

Skip – List
–Delete Element
–Space Complexity Analysis

Program :

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
```

```

        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
            lst->level--;
        }
        cout << "Successfully deleted key " << key << endl;
    } else {
        cout << "Key " << key << " not found. Cannot delete." << endl;
    }
}

...
case 2:
    cout << "Enter key to delete: ";
    cin >> key;
    deleteElement(list, key);
    break;

```

Space Complexity Analysis

1. Input Space Complexity

A. Parametric Input Space: $O(1)$

- The ***deleteElement*** function signature requires exactly two arguments: a pointer to the Skip List (***lst***) and the integer to be deleted (***key***).
- Because these take a fixed, tiny amount of memory (***typically 8 bytes and 4 bytes***) regardless of how many millions of nodes exist in the list, the parametric input space is strictly ***$O(1)$*** .

B. Full Data Holistic Input Space: $O(n)$

- *The function operates on an already existing Skip List containing n nodes in memory.*
- *Because the probabilistic coin flip ensures the average number of pointers per node remains a small constant (e.g., 2 pointers when $p = 0.5$), the total footprint of the list scales linearly with the number of inserted elements.*
- *Therefore, the full data holistic input space is $O(n)$.*

2. Auxiliary Space Complexity: $O(1)$ Practical

/ $O(\log n)$ Theoretical

- ***The Array Allocation:*** *To remember this path, the function explicitly declares an array: `struct Node * update[MAX_LEVEL + 1];`.*
- ***Practical Scenario ($O(1)$):*** *In your specific C++ program, `MAX_LEVEL` is a hardcoded constant (e.g., 3). Because the array is always strictly capped at 4 slots (32 bytes) no matter how large the list grows, the practical auxiliary space is exactly $O(1)$.*

In a real – world software implementation, `MAX_LEVEL` is not calculated dynamically based on n while the program runs.

It is defined as a fixed hard – coded limit based on the maximum anticipated capacity of the data structure (e.g., `Max_Level = 32 or 64`).

- *Theoretical Scenario ($O(\log n)$): In a purely mathematical, infinitely scaling Skip List, the maximum level H dynamically scales to $\log_{1/p}(n)$. Because the *update* array must have enough slots to hold a pointer for every possible floor, the size of the array scales logarithmically. Therefore, the theoretical auxiliary space is $O(\log n)$.*

Conclusion:

When asked in a standard academic or interview setting, the auxiliary space for a Skip List deletion algorithm is generally cited as $O(\log n)$ because it requires tracking the logarithmic height of the search path to safely rewire the pointers.

Algorithm: SkipListDelete(LST, KEY)

1. [Initialize Pointers and Arrays]

SET CURRENT := HEADER[LST].

Declare an array UPDATE of size MAX_LEVEL

+ 1 and initialize all elements to NULL (or 0).

2. [Traverse List to Record Path]

Repeat for i = LEVEL[LST] down to 0:

*Repeat while FORWARD[i][CURRENT] ≠ NULL AND
INFO[FORWARD[i][CURRENT]] < KEY:*

SET CURRENT := FORWARD[i][CURRENT].

[End of while Loop]

SET UPDATE[i] := CURRENT.

[End of for Loop]

3. [Move to Target Node]

SET CURRENT := FORWARD[0][CURRENT].

4. [Verify Key Existence]

If CURRENT ≠ NULL AND INFO[CURRENT] = KEY, Then:

Proceed to Step 5.

Else:

Proceed to Step 9.

5. [Rewire Pointers to Bypass Node]

Repeat for i = 0 to LEVEL[LST]:

If FORWARD[i][UPDATE[i]] ≠ CURRENT, Then:

*BREAK loop (If a level doesn't point to current,
higher levels won't either).*

[End of If Structure]

SET FORWARD[i][UPDATE[i]] := FORWARD[i][CURRENT].

[End of for Loop]

6. [Free Memory]

CALL free(FORWARD[CURRENT]).

CALL free(CURRENT).

7. [Adjust Global List Level]

*Repeat while LEVEL[LST] > 0 AND FORWARD[LEVEL[LST]][HEADER[LST]]
= NULL:*

SET LEVEL[LST] := LEVEL[LST] - 1.

[End of while Loop]

8. [Success Message]

Write: "Successfully deleted key ", KEY.

Proceed to Step 10.

9. [Handle Key Not Found]

Write: "Key ", KEY, " not found. Cannot delete."

10. [Exit]

END OF ALGORITHM.
